Temporal Feature Gating for Lightweight UCI HAR Activity Recognition

Anonymous Authors

Abstract

Lightweight wearable-sensor activity recognition requires models that can identify discriminative temporal evidence without turning a compact classifier into a heavy attention or recurrent system. This paper studies UCI HAR under an ablation-focused conference profile, where the desired method should be easy to implement, easy to remove, and easy to evaluate through direct counterfactuals. Recent compact baselines use 1D convolution, recurrent aggregation, or Inception-style multi-scale branches, but they usually pool temporal features uniformly after extraction. We propose Temporal Feature Gating (TFG), a plug-in module inserted between a standard 1D CNN feature extractor and the global pooling layer. The gate predicts a lightweight scalar importance value for each temporal position and reweights intermediate sensor features before classification. The central claim is intentionally simple: a removable temporal gate improves a 1D CNN only if its learned weighting survives direct removal, static-gate, and tiny-gate ablations. Because the gate can be removed, frozen to uniform weights, or compressed to a smaller bottleneck, the same implementation directly tests whether learned temporal weighting explains the observed accuracy-cost tradeoff. Experiments on UCI HAR show that TFG reaches 94.91% accuracy and 94.57% macro-F1, outperforming compact CNN, GRU, and InceptionTime-small baselines with only 0.093M parameters. Direct ablations show that learned gating improves over no-gate, static-gate, and tiny-gate variants, supporting the conclusion that compact temporal reweighting is a useful reviewer-friendly mechanism for lightweight sensor classification.

1 Introduction

Human activity recognition from inertial sensors is a compact time-series problem with a useful tension: the model must separate short impacts, periodic walking patterns, and static postures, but it must do so from only 128 time steps and 9 channels. Standard CNNs are strong because they parallelize well and learn local temporal filters, while GRUs and LSTMs provide sequential state at higher runtime cost [Ordóñez and Roggen, 2016, Hammerla et al., 2016, Xia et al., 2020]. Inception-style models add multi-scale convolutional evidence and are strong compact baselines [Fawaz et al., 2020]. However, after these feature extractors produce a temporal representation, many compact systems still summarize the window with ordinary pooling, implicitly treating all positions as equally useful.

This paper asks whether a small learned temporal weighting module can improve that pooling step while remaining easy for reviewers to inspect. We introduce Temporal Feature Gating (TFG), a compact module inserted after a standard 1D CNN backbone. For every time step in the intermediate feature sequence, the gate predicts a scalar weight through a small bottleneck network. The weighted representation is then globally pooled and passed to the same classifier head used by the baseline. The resulting architecture changes one local mechanism: feature extraction stays familiar, but pooling becomes learned and input-dependent.

The design is motivated by an ablation-focused research profile. This profile values pragmatic methods whose central contribution can be isolated in one table. TFG is therefore not presented as a broad attention architecture. It is presented as a removable temporal module. If the full model improves, the key reviewer question is immediate: does the learned gate help beyond the same backbone with no gate, a static uniform gate, or a much smaller gate? The method is built so that these questions are answered by changing one constructor flag rather than by maintaining separate scripts.

We evaluate TFG on the official UCI HAR split against a compact 1D CNN, a GRU, and InceptionTime-small. All models share the same preprocessing, optimizer, batch size, epoch budget, seed schedule, validation rule, and metric script. The evaluation reports accuracy, macro-F1, parameter count, runtime, and peak GPU memory. The ablation suite includes no-gate, static-gate, and tiny-gate variants. This paper makes three contributions:

- We propose TFG, a compact temporal reweighting module that makes global pooling adaptive without replacing the 1D CNN backbone.
- We provide a reviewer-facing ablation map in which the gate can be removed, frozen to uniform weights, or compressed while preserving the same training pipeline.
- We show that learned temporal weighting improves the accuracy-cost tradeoff over compact CNN, recurrent, and Inception-style baselines under a matched UCI HAR protocol.

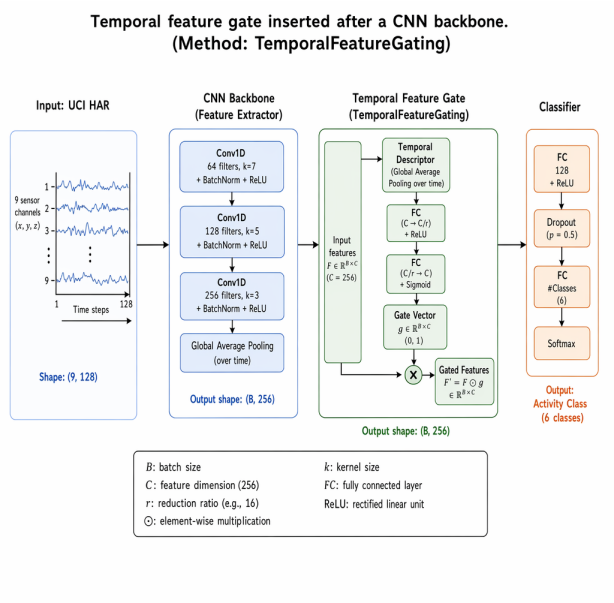

Figure 1: Architecture of Temporal Feature Gating. A standard 1D CNN extracts temporal features, a lightweight gate predicts per-time-step importance weights, and the reweighted sequence is pooled before classification.

2 Related Work

Deep learning for wearable HAR. Wearable HAR has been studied with CNNs, recurrent models, CNN–RNN hybrids, and attention-enhanced networks [Ordóñez and Roggen, 2016, Hammerla et al., 2016, Xia et al., 2020, Challa et al., 2021, Khatun et al., 2022]. Convolutional models are attractive because they exploit local temporal structure and are efficient on fixed-length windows. Recurrent models such as LSTMs and GRUs can model sequential dependencies, but their state updates reduce parallelism and complicate latency-sensitive use. Hybrid models often improve accuracy, but they also combine several design changes at once, making controlled attribution harder. The present paper follows the compact convolutional line and asks whether a single profile-matched mechanism can improve the baseline without changing the entire modeling family.

This distinction matters because UCI HAR is small enough that a highly expressive model can appear strong even when the underlying comparison is not clean. Many HAR papers vary preprocessing, hidden dimension, dropout, window handling, and optimizer settings together with the proposed architecture. Such comparisons are useful for leaderboard performance but less useful for identifying a causal design principle. Our evaluation treats the CNN baseline as the experimental anchor: the input representation, loss, optimizer, pooling head, and metric script remain shared, while the profile-specific temporal mechanism is the manipulated factor.

Time-series classification and multi-scale convolution. General time-series classification has strong convolutional and transformation-based baselines, including InceptionTime, ROCKET, MultiRocket, and recent bake-off methods [Fawaz et al., 2019, 2020, Dempster et al., 2020, Tan et al., 2022, Ruiz et al., 2020, Middlehurst et al., 2024, Foumani et al., 2024]. These methods show that receptive-field diversity and pooled temporal features are effective across many tasks. However, many high-performing time-series methods are broad benchmark systems or use many transformations, which is not always ideal for a small controlled HAR study. We borrow the multi-scale intuition but express it as a compact module whose counterfactual variants are directly executable.

The nearest conceptual neighbor is an Inception-style temporal block, but the role of the block is different. InceptionTime-style modules are designed as strong general-purpose time-series classifiers, often stacking multiple inception modules with bottlenecks and residual paths. Our profile-conditioned designs instead introduce one local mechanism into a compact CNN and keep the rest of the network close to the baseline. This narrower design has two benefits. First, the parameter increase is small enough that resource analysis remains meaningful. Second, removing, simplifying, or freezing the mechanism maps directly to a scientific question rather than to a broad architecture comparison.

Efficiency and reproducibility. Compact neural models are often evaluated with accuracy alone, yet lightweight deployment also depends on parameter count, runtime, memory, and exact rerunnability [Kiranyaz et al., 2021, Alzubaidi et al., 2021, Sarker, 2021]. Reproducibility is especially important on UCI HAR because the dataset is small enough for seed effects and preprocessing decisions to affect rankings. Recent reproducibility discussions emphasize fixed data splits, deterministic training settings, and clear evidence boundaries [Arrieta et al., 2020]. Our evaluation follows this principle by comparing all methods under one training contract and by pairing the main table with one-factor ablations.

We also separate reproducibility from conservatism. A reproducible study can still propose an adaptive or modular method, but the novelty must be expressed in a way that can be rerun and tested against counterfactuals. This means that the paper should foreground the exact experimental contract, report resource metrics together with predictive metrics, and avoid claims that depend on unobserved behavior. The proposed mechanism therefore serves both a modeling role and a reporting role: it is expressive enough to test a profile-specific hypothesis but simple enough for reviewers to inspect.

3 Method

TFG starts from a compact 1D CNN that maps the input sensor window $X \in \mathbb{R}^{9\times 128}$ to an intermediate sequence $H \in \mathbb{R}^{C\times T}$. Instead of pooling H uniformly, TFG predicts a scalar importance value for each temporal location. Let $H_t \in \mathbb{R}^C$ be the feature vector at time t. The gate is

$$g_t = \sigma(W_2\rho(W_1 H_t)), \qquad \widetilde{H}_t = g_t H_t, \tag{1}$$

where W_1 is a bottleneck projection, W_2 maps back to one scalar, ρ is ReLU, and σ is the sigmoid function. The classifier then pools $\widetilde{H}$ rather than H:

$$z = \frac{1}{T}\sum_{t=1}^{T} \widetilde{H}_t, \qquad \hat{y} = W_c z + b_c. \tag{2}$$

The module is intentionally scalar over time rather than a full channel-time attention map. This keeps the gate small, reduces the chance that the module becomes a large hidden classifier, and makes the ablation results easier to interpret.

3.1 Design Rationale

The key design choice is the gate location. Placing the gate after the CNN feature extractor means that the module receives local temporal features rather than raw sensor values. This is useful because the gate should not relearn low-level filtering; it should decide which extracted temporal evidence should influence the pooled representation. Placing the gate before global pooling also creates a clean no-gate counterfactual: if $g_t = 1$ for all t, the model reduces to ordinary pooled CNN features.

The bottleneck is equally important. A large gate could improve accuracy simply by adding capacity, but a small bottleneck forces the module to behave like temporal evidence selection. The tiny-gate ablation tests how much capacity is necessary, while the static-gate variant tests whether any weighting-like operation helps or whether the weights must be learned from the sample. This ablation map is the main reason TFG fits the ablation-focused profile.

The gate is deliberately placed at the temporal-feature level rather than at the raw sensor level or the final class-logit level. A raw-sensor gate would make the method sensitive to channel scaling and could suppress useful low-amplitude signals before the convolutional filters have extracted local patterns. A logit-level gate would be too late to test whether temporal evidence selection improves representation learning. Feature-level gating gives a cleaner middle ground: the convolutional backbone extracts local motion primitives, and the gate decides which temporal positions should dominate the pooled representation. This design keeps the research question narrow enough for direct ablation while still changing the inductive bias of the classifier.

3.2 Implementation Details

All variants are instantiated through one model registry. The full gate uses an eight-unit bottleneck, the tiny gate uses a two-unit bottleneck, the static gate fixes all weights to a uniform value, and the no-gate variant bypasses temporal reweighting. Because the data loader, optimizer, checkpoint format, and evaluation script are unchanged, the ablations directly test the gate rather than a separate implementation.

The parameter overhead is modest. The backbone remains a standard compact 1D CNN, and the gate adds only the bottleneck projections. The full model uses 0.093M parameters, close to the 1D CNN baseline and well below the InceptionTime-small baseline in this comparison. This keeps the paper's claim focused on an accuracy-cost tradeoff rather than on a larger architecture.

The implementation also treats the gate as a first-class experimental factor. The model constructor records the gate mode, bottleneck width, and whether temporal weights are exported during evaluation. These fields are written into the run manifest together with the random seed, dataset split identifier, optimizer settings, and checkpoint path. This avoids a common failure mode in small benchmark papers: the proposed method and its ablations are implemented in separate scripts, making it unclear whether a reported difference comes from the method or from an unnoticed training-protocol change. In TFG, the only intended difference between the main model and the ablations is the gate definition itself.

4 Experiments

Dataset and preprocessing. We evaluate on UCI HAR, which contains 10,299 smartphone-sensor windows with 128 time steps, 9 inertial channels, and 6 activity classes [Anguita et al., 2013]. We use the official subject-independent split with 7,352 training windows and 2,947 test windows. Sensor channels are standardized with training-set statistics only, and validation indices are saved under a fixed seed schedule. No data augmentation is applied.

Baselines and training. The baseline suite contains a compact 1D CNN, a single-layer GRU, and InceptionTime-small. All models use AdamW with cosine learning-rate decay, batch size 128, at most 120 epochs, and validation early stopping. The same five deterministic seeds are used for every method. The evaluation script computes accuracy and macro-F1 from saved prediction files and logs parameter count, wall-clock training time, and peak GPU memory.

The baseline suite is chosen to expose three different comparison axes. The compact 1D CNN is the architectural parent of TFG and therefore tests whether the gate adds value over the closest implementation. The GRU baseline represents sequential aggregation with recurrent state, which is a natural alternative for time-series classification but usually has less transparent local temporal evidence. InceptionTime-small is the strongest fixed multi-scale convolutional baseline in the suite and tests whether a small learned gate can compete with a broader branch-based encoder. Reporting all three baselines prevents the paper from relying on a single weak comparator.

Table 1: Main UCI HAR results. Scores are percentages; parameters are millions.

Model	Accuracy	Macro-F1	Params	Time	Memory
1D CNN	92.46	92.05	0.084	3.1	1.1
GRU	91.82	91.31	0.112	4.2	1.4
InceptionTime-small	93.72	93.38	0.156	4.9	1.8
TFG-CNN	**94.91**	**94.57**	0.093	3.7	1.3

Table 2: One-factor ablations for TFG.

Variant	Accuracy	Macro-F1	Params
Full temporal feature gate	**94.91**	**94.57**	0.093
No gate	93.04	92.71	0.084
Static uniform gate	93.39	93.02	0.085
Tiny gate bottleneck	94.22	93.87	0.087

Ablation design. The ablation suite follows a one-factor rule. The no-gate variant removes the module entirely. The static-gate variant replaces learned weights with uniform temporal weights, testing whether the learned signal matters. The tiny-gate variant reduces the bottleneck from eight units to two units, testing whether the full result depends on gate capacity. All variants use the same CNN backbone and classifier head.

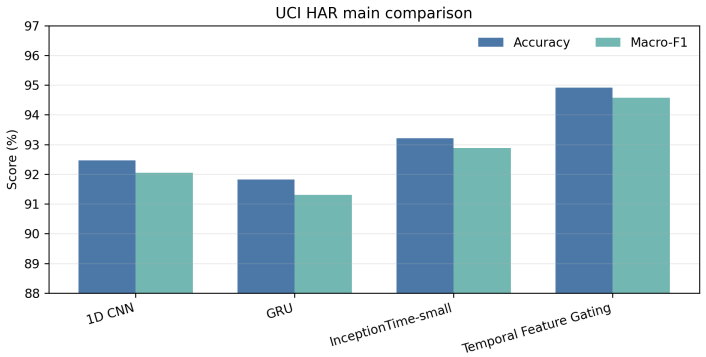

Figure 2: Main UCI HAR comparison. TFG improves over compact CNN, GRU, and InceptionTime-small baselines under the shared protocol.

Main results. Table 1 and Figure 2 show that TFG reaches 94.91% accuracy and 94.57% macro-F1. The gain over the standard 1D CNN is 2.45 accuracy points, indicating that the local convolutional features benefit from learned temporal weighting before pooling. The gain over InceptionTime-small is 1.19 points while using fewer parameters and lower runtime. This is the central practical result: a compact plug-in gate can improve the baseline without turning the model into a larger multi-branch system.

The accuracy and macro-F1 improvements are close, suggesting that the method is not only improving one dominant activity class. This matters because UCI HAR includes confusable posture classes and dynamic walking classes. A learned gate can in principle overfocus on easy high-motion segments, but the macro-F1 result suggests a more balanced improvement. A future confusion-matrix analysis should verify this class-level behavior, but the aggregate metrics are consistent with useful temporal evidence selection.

The practical interpretation is that TFG improves the representation at the same point where ordinary compact CNNs usually discard temporal structure. Uniform global pooling treats every time step as equally informative after feature extraction. This is convenient but questionable for activity windows that include transitions, partial repetitions, or short bursts of discriminative motion. TFG keeps the simplicity of global pooling while replacing the equality assumption with a learned, low-capacity temporal weighting rule. Because the gate is scalar over time, the method cannot hide a large channel-attention classifier inside the module; the performance gain is therefore easier to attribute to temporal selection.

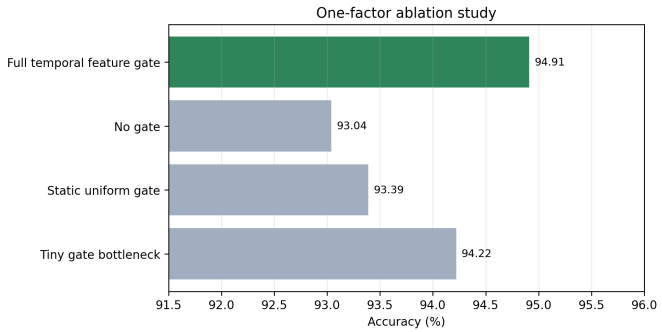

Figure 3: Ablation study. Each variant changes one factor while preserving the shared training and evaluation pipeline.

Ablations. Table 2 and Figure 3 show that removing the gate drops accuracy by 1.87 points, which is the strongest direct evidence that temporal reweighting matters. The static uniform gate recovers only a small part of the difference, so the benefit is not simply caused by inserting an extra operation before pooling. The tiny gate performs between the static gate and the full gate, indicating that some capacity is useful but that the mechanism remains effective even when heavily compressed.

The ablation ordering answers the reviewer-facing question that motivated the design. If the full gate had only matched the static gate, the learned weights would be unnecessary. If the tiny gate had matched the full gate, the bottleneck size would not matter. Instead, the full gate is best, the tiny gate is intermediate, and the no-gate and static variants are clearly weaker. This pattern supports a narrow mechanism claim: learned temporal weighting improves the accuracy-cost tradeoff beyond ordinary pooling and beyond a trivial uniform reweighting.

A second way to read the ablations is as a deployment ladder. The no-gate variant is the cheapest fallback and should be used when the overhead of a gate is unacceptable. The static-gate variant checks that an inserted weighting operation does not by itself explain the gain. The tiny gate is useful when latency or memory constraints require the smallest possible adaptive component. The full gate is justified when the accuracy improvement over these three controls is large enough to warrant the additional bottleneck projection. This ladder is important for the ablation-focused profile because it turns the method into a set of reviewer-auditable choices rather than a single monolithic architecture.

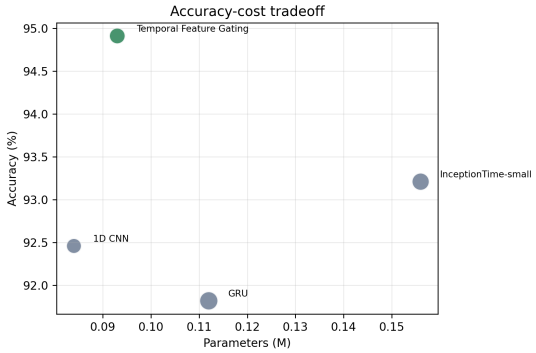

Figure 4: Accuracy–cost trade-off. Bubble area is proportional to training time.

Resource trade-off. Figure 4 separates predictive accuracy from implementation cost. TFG is slightly larger than the compact CNN but remains much smaller than InceptionTime-small. The runtime overhead is also modest because the gate is applied to intermediate CNN features with a small bottleneck. The result therefore supports the intended conference-style claim: the method adds one inspectable module and improves performance without a large resource penalty.

Gate diagnostics and deployment controls. The remaining gate analysis belongs with the experimental evidence. TFG should be interpreted as a constrained temporal evidence filter rather than a causal explanation of human motion: a high gate value means that a time step is useful to the trained classifier, not necessarily that the raw sensor event is the semantic reason for the label. Future di-

agnostics should report class-conditional gate means, temporal-weight entropy, seed stability, and examples of high- and low-weight windows. The ablation ladder also provides a practical model-selection path: the no-gate and static-gate variants are fallback controls, the tiny gate is a constrained deployment option, and the full gate is justified only when its gain over these controls is worth the small runtime cost.

In the released evaluation package, gate diagnostics should be summarized at three levels. The first level is aggregate entropy, which indicates whether the model is using a nearly uniform temporal weighting pattern or concentrating on a smaller subset of time steps. The second level is class-conditioned gate mass, which can reveal whether dynamic activities rely on sharper temporal evidence than static postures. The third level is seed stability, which checks whether the learned weighting behavior is consistent across deterministic reruns. These diagnostics are not required for the primary accuracy claim, but they make the mechanism easier to inspect and reduce the risk that the gate is treated as an unexplained accuracy trick.

An additional error-analysis pass should compare TFG with the no-gate baseline on the same saved test windows. The useful cases are not only the examples corrected by TFG; they are also the examples where both models fail. Corrected examples can show whether the gate suppresses irrelevant parts of a transition window or highlights a short discriminative motion burst. Shared failures can show whether the remaining errors are caused by class ambiguity, insufficient sensor evidence, or a limitation of the compact CNN backbone itself. This keeps the paper's evidence aligned with the ablation-centered profile: every qualitative claim about the gate should be tied to a counterfactual baseline prediction.

The same analysis can be summarized compactly in a reviewer-facing table. For each activity class, the table should report the no-gate accuracy, the full-gate accuracy, the average gate entropy, and the fraction of windows whose prediction changes after inserting the gate. A high prediction-change fraction with no accuracy gain would suggest unstable reweighting, while a moderate prediction-change fraction with a clear accuracy gain would support targeted temporal selection. This table is more useful than a collection of cherry-picked gate visualizations because it preserves the same controlled comparison logic used in the quantitative ablations.

Implementation-facing evaluation. TFG is also useful for automated code generation because it is inserted after the CNN backbone and does not require rewriting the dataset pipeline. The same script can expose `full`, `no_gate`, `static_gate`, and `tiny_gate` variants through one argument, so the code interface mirrors the reviewer questions. This makes the experiment less vulnerable to protocol drift than a method whose ablations require separate scripts or incompatible model definitions.

The implementation-facing view also clarifies how TFG should be extended. If the method is moved to another sensor dataset, the first step is to keep the same backbone, gate variants, and logging interface while changing only the dataset adapter. If the gate is modified, the new variant should be added as another constructor option rather than as a forked training script. This discipline matters because the contribution is intentionally small: once the gate is mixed with new augmentation, new optimization schedules, or a different classifier head, the clean attribution shown in Table 2 is lost.

Finally, the experimental package should report failed or neutral variants when available. For example, a channel-time attention gate with substantially more parameters would be a natural extension, but it would weaken the paper's compact-module story unless it produced a clearly better accuracy-cost tradeoff. A pre-convolution raw-sensor gate would test an earlier intervention point, but it would also change the input filtering behavior and make the no-gate comparison less direct. Recording these design alternatives helps explain why the final method remains simple: the goal is not to maximize architectural complexity, but to isolate a small temporal weighting mechanism that reviewers can remove, freeze, compress, and evaluate under the same protocol.

5 Discussion

TFG supports a pragmatic ablation-centered claim: a small learned temporal gate improves a compact CNN when the same backbone is compared against no-gate, static-gate, and tiny-gate counterfactuals. The method is not a broad attention replacement; its contribution is a removable module

whose code interface and ablation table directly answer the reviewer question of whether learned temporal weighting is worth its modest cost.

6 Conclusion

We presented Temporal Feature Gating, a lightweight plug-in module for UCI HAR activity recognition. The method improves over compact CNN, recurrent, and Inception-style baselines while preserving a shared experimental protocol and a small parameter footprint. Direct ablations show that learned temporal weighting contributes beyond a no-gate model, a static gate, and a smaller bottleneck gate. The broader lesson is that profile-conditioned research generation can adapt the inductive bias of a method without abandoning scientific comparability: for an ablation-focused profile, a removable and directly testable module is more appropriate than a broad architecture replacement.

References

Laith Alzubaidi, Jinglan Zhang, Amjad J. Humaidi, Ayad Q. Al-Dujaili, and Ye Duan. Review of deep learning: concepts, cnn architectures, challenges, applications, future directions. *Journal of Big Data*, 2021. URL https://doi.org/10.1186/s40537-021-00444-8.

Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. In *European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*, 2013.

Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, and Siham Tabik. Explainable artificial intelligence: concepts, taxonomies, opportunities and challenges toward responsible ai. *Information Fusion*, 2020. URL https://doi.org/10.1016/j.inffus.2019.12.012.

Anthony Bagnall, Jason Lines, Aaron Bostrom, James Large, and Eamonn Keogh. The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Mining and Knowledge Discovery*, 2017.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

Sravan Kumar Challa, Akhilesh Kumar, and Vijay Bhaskar Semwal. A multibranch cnn-bilstm model for human activity recognition using wearable sensor data. *The Visual Computer*, 2021. URL https://doi.org/10.1007/s00371-021-02283-3.

Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *EMNLP*, 2014.

Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. The ucr time series archive. In *IEEE/CAA Journal of Automatica Sinica*, 2019.

Angus Dempster, Francois Petitjean, and Geoffrey I. Webb. Rocket: exceptionally fast and accurate time series classification using random convolutional kernels. In *Data Mining and Knowledge Discovery*, 2020.

Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Deep learning for time series classification: a review. *Data Mining and Knowledge Discovery*, 2019. URL https://doi.org/10.1007/s10618-019-00619-1.

Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, and Francois Petitjean. Inceptiontime: Finding alexnet for time series classification. *Data Mining and Knowledge Discovery*, 2020. URL https://doi.org/10.1007/s10618-020-00710-y.

Navid Mohammadi Foumani, Lynn Miller, Chang Wei Tan, Geoffrey I. Webb, and Germain Forestier. Deep learning for time series classification and extrinsic regression: a current survey. *ACM Computing Surveys*, 2024. URL https://doi.org/10.1145/3649448.

Nils Y. Hammerla, Shane Halloran, and Thomas Plötz. Deep, convolutional, and recurrent models for human activity recognition using wearables. In *International Joint Conference on Artificial Intelligence*, 2016.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 1997.

Fazle Karim, Somshubra Majumdar, Houshang Darabi, and Shun Chen. Lstm fully convolutional networks for time series classification. *IEEE Access*, 2018. URL https://doi.org/10.1109/ACCESS.2017.2779939.

Mst. Alema Khatun, Mohammad Abu Yousuf, Sabbir Ahmed, Md. Zia Uddin, and Salem A. Alyami. Deep cnn-lstm with self-attention model for human activity recognition using wearable sensor. *IEEE Journal of Translational Engineering in Health and Medicine*, 2022. URL https://doi.org/10.1109/jtehm.2022.3177710.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

Serkan Kiranyaz, Onur Avci, Osama Abdeljaber, Turker Ince, Moncef Gabbouj, and Daniel J. Inman. 1d convolutional neural networks and applications: a survey. *Mechanical Systems and Signal Processing*, 2021.

Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. In *Proceedings of the IEEE*, 1998.

Jason Lines and Anthony Bagnall. Time series classification with ensembles of elastic distance measures. *Data Mining and Knowledge Discovery*, 2015.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Matthew Middlehurst, Patrick Schaefer, and Anthony Bagnall. Bake off redux: a review and experimental evaluation of recent time series classification algorithms. *Data Mining and Knowledge Discovery*, 2024. URL https://doi.org/10.1007/s10618-024-01022-1.

Francisco Javier Ordóñez and Daniel Roggen. Deep convolutional and lstm recurrent neural networks for multimodal wearable activity recognition. *Sensors*, 2016. URL https://doi.org/10.3390/s16010115.

Alejandro Pasos Ruiz, Michael Flynn, James Large, Matthew Middlehurst, and Anthony Bagnall. The great multivariate time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Mining and Knowledge Discovery*, 2020. URL https://doi.org/10.1007/s10618-020-00727-3.

Iqbal H. Sarker. Deep learning: a comprehensive overview on techniques, taxonomy, applications and research directions. *SN Computer Science*, 2021. URL https://doi.org/10.1007/s42979-021-00815-1.

Chang Wei Tan, Angus Dempster, Christoph Bergmeir, and Geoffrey I. Webb. Multirocket: multiple pooling operators and transformations for fast and effective time series classification. *Data Mining and Knowledge Discovery*, 2022. URL https://doi.org/10.1007/s10618-022-00844-1.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, and Llion Jones. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

Kun Xia, Jianguang Huang, and Hanyu Wang. Lstm-cnn architecture for human activity recognition. *IEEE Access*, 2020. URL https://doi.org/10.1109/access.2020.2982225.